

[← Go Back](#)

Hardware

Portenta Machine Control ▼

Tutorials

Memory Partitioning for Use with the Arduino IDE

Arduino Portenta Machine Control Library Guide

Modbus RTU On Portenta Machine Control Using PLC IDE

Modbus TCP On Portenta Machine Control Using PLC IDE

Modbus TCP with Portenta Machine Control & Opta™

Tank Thermoregulation with Portenta Machine Control & Opta™

Connect an RTD/Thermocouple to the Portenta Machine Control

Portenta Machine Control User Manual

Home / Hardware / Portenta Machine Control / **Arduino Portenta Machine Control Library Guide**

Arduino Portenta Machine Control Library Guide

Learn about the new Arduino Portenta Machine Control library, which enables efficient management of the features of the Arduino Portenta Machine Control.

Author · José Bagur · Last revision · 12/02/2024

Overview

The Arduino Portenta Machine Control is a versatile industrial control unit for driving machinery. It offers soft-PLC control, diverse input/output options, and flexible network connectivity. The

Arduino_PortentaMachineControl

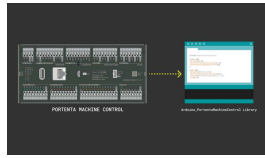
library enables efficient management of the software and hardware features of the Portenta Machine Control. This library is an upgraded and improved version designed to replace the deprecated **Arduino_MachineControl** library. The tutorial aims to highlight the recent changes in the library, focusing on its user-

ON THIS PAGE

Overview

- Goals
- Hardware and Software Requirements +
- The Arduino Portenta Machine Control Library +
- Key Major Improvements +
- Conclusion

ecosystem rules, conventions, and style.



The Arduino ecosystem: hardware and software

Goals

Introduce the

`Arduino_PortentaMachineControl`

library's latest release key features and capabilities.

Show the main differences between the

`Arduino_PortentaMachineControl`

library and the deprecated

`Arduino_MachineControl`

.

Learn how to port a program using the old library to the new one.

Hardware and Software Requirements

Hardware Requirements

Portenta Machine Control (x1)

USB-A to Micro-USB

The minimum recommended output current rating for the Portenta Machine Control +24 VDC power supply is 0.5 A



Software Requirements

[Arduino IDE 2.0+](#) or
[Arduino Cloud Editor](#)

[Arduino_PortentaMachineControl](#) library

The Arduino Portenta Machine Control Library

The

```
Arduino_PortentaMachineControl
```

is a C++ library designed to manage the software and hardware functionalities of the Portenta Machine Control board efficiently. It provides extensive support for inputs such as digital, analog, and encoder ports while offering outputs including digital and analog ports. This library also manages communication through interfaces like CAN bus, RS-232, RS-422, and RS-485, allowing connectivity via Ethernet, Wi-Fi®, Bluetooth® Low Energy and USB.

The library empowers users to easily initialize, control, and

functionalities of the Portenta Machine Control, enhancing its capability and adaptability for industrial applications.

Library Features

The

```
Arduino_PortentaMachineControl
```

library offers a wide range of functionalities organized for ease of reference:

Input signals

Eight digital inputs at +24 VDC

Two channels for encoder readings

Three analog inputs for PT100, J, and K temperature probes

Three analog inputs for 4-20 mA, 0-10V, and NTC signals

Output signals

Eight digital output terminals at +24 VDC

Four analog output terminals, ranging from 0 to 10 VDC

Programmable I/O

12 programmable digital input/output terminals at +24 VDC

Communication protocols

CAN bus

Serial protocols (RS-232, RS-422, and RS-485)

USB

Improvements Between Libraries

Structural and Functional Enhancements

The library has undergone significant structural changes to improve its efficiency and user-friendliness. Some of these key improvements include the following:

Library structure

revamping: Transition from a single header file to multiple `.cpp` and `.h` files organized by functionality (check [Library Features](#) section)

Naming refactoring:

Removal of the

```
machinecontrol  
namespace
```

, adopting a more descriptive prefix-based naming system.

Function optimization:

Unused functions have been removed, streamlining the library.

Direct calls to mbed core

removed: This change enhances the library's compatibility and ease of use.

Examples reworking:

Updated examples provide clearer demonstrations of the library's capabilities.

Documentation

improvement: Enhanced documentation offers better support to users.

Practical Application Enhancements

The following improvements make the Portenta Machine Control Library more adaptable and efficient for various industrial applications, from machinery control to sensor integration:

Input/output

management: Enhanced handling of digital and analog signals for improved accuracy and reliability.

Communication protocol

integration: Improved support for CAN-BUS and serial protocols, facilitating smoother data exchange.

RTC functionality: More robust RTC management, ensuring precise timekeeping for time-sensitive applications.

Key Major Improvements

Naming Conventions

One of the significant updates in the Portenta Machine Control Library is the change in **naming conventions**, which enhances clarity and consistency across the library.

Changes in naming:

Removal of

`machinecontrol`

namespace: The previous

namespace. This has been removed to simplify the code and make it more accessible.

Introduction of the

`MachineControl_` prefix:

In the updated library, a prefix `MachineControl_` is used for each functionality's singleton object. This change ensures a clearer and more descriptive approach to accessing library functionalities.

Example of the Updated Naming

Old naming convention:

```
1 // Using the 'machined
2 using namespace machin
3 void setup() {
4     digital_inputs.ini
5 }
```

New naming convention:

```
1 // Prefix 'MachineCont
2 void setup() {
3     MachineControl_Dig
4 }
```

This new naming structure aligns with the Arduino programming language standard of using clear and descriptive names, making the library more user-friendly, especially for those new to

Input/Output Modules

In the

```
Arduino_PortentaMachineControl
```

Library, interacting with input/output modules has been streamlined for better usability and consistency with Arduino standards. Two new important features are the following:

Initializing input/output modules:

Initializing an input/output module is more intuitive in the updated library. The `begin()` function is now used to start a module, providing a clear starting point for module operations.

CamelCase notation: The library has adopted CamelCase notation for function names, aligning with the standard Arduino library style. This change makes the library more consistent with other Arduino libraries, enhancing readability and ease of use.

Example of the Input/Output Module Usage:

Old approach:

```
1 // Using the previous
2 #include <Arduino_Machin
3 using namespace machin
4 void setup() {
5     analog_out.period
6 }
```



```
1 // Using the updated I
2 #include <Arduino_Por
3 void setup() {
4     MachineControl_AnalogOut
5     MachineControl_AnalogIn
6 }
```

In the new approach, the `begin()` function is explicitly called to initialize the `MachineControl_AnalogOut` module. Following the CamelCase notation, the `setPeriod()` method replaces the older `period_ms()` function, making the function's purpose clearer and the code more consistent with Arduino standards.

This update enhances the user experience by making the code more intuitive and aligned with the familiar Arduino programming style.

When using the Analog Output channels, please consider following detail: The High-Resolution Timer (HRTIM) function on PG7 allows for high-frequency PWM signals with precise control over short periods. However, it is limited to a maximum period of **approximately 1.3 ms**. This constraint makes **Analog Out**



Channel 2 optimal for high-frequency PWM applications but unsuitable for periods exceeding 1.3 ms. For longer periods, alternative **Analog Out channels (AO0, AO1, AO3)** with standard timers are recommended. Please refer to the [Portenta Machine Control User Manual: Analog Outputs section](#) for more details.

Encoders

The handling of encoders has been updated in the

```
Arduino_PortentaMachineControl
```

Library for a more streamlined and intuitive approach. One key new feature is the **removal of array object access**. In the previous library version, encoder objects were accessed using array notation (`[ ]`). The latest version has simplified this approach, enhancing code readability and reducing potential errors.

Example of Encoder Usage

Old approach:

```
1 // Using the previous
2 #include <Arduino_Mach
3 using namespace machi
4 int rev;
5 void setup() {
6     rev = encoders[0];
7 }
```

New approach:

```
1 // Using the updated I
2 #include <Arduino_Por
3 int rev;
4 void setup() {
5     rev = MachineContr
6 }
```

In the new approach, the `getRevolutions()` function is called directly on the `MachineControl_Encoders` object rather than accessing an encoder array. The encoder of interest is specified as a parameter in the function call, making the code more explicit and easier to understand.

This update to the encoder module in the Portenta Machine Control Library simplifies the interaction with encoders, making it more straightforward for users to implement in their projects.

Communication Interfaces

The latest Portenta Machine Control Library update has refined how communication interfaces are managed. The updated library version moves away from using public objects for communication protocols; this change leads to a more encapsulated and modular approach, enhancing the robustness and clarity of the communication code.

Example of Communication Protocol Usage

Let's use the **RS-485 interface** as example to show the new communication protocol usage.

Old approach:

```
1 // Using the previous
2 #include <Arduino_Mach
3 using namespace machir
4 void setup() {
5     comm_protocols.ini
6     comm_protocols.rs4
7     comm_protocols.rs4
8 }
```

New approach:

```
1 // Using the updated I
2 #include <Arduino_Portenta_
3 void setup() {
4     MachineControl_RS4
5     MachineControl_RS4
6 }
```

The new approach directly handles communication initialization and data reception through the

MachineControl_RS485Comm object

. This design eliminates the need to initialize a separate communication protocol object (`comm_protocols`) and then access its members, making maintaining the code more straightforward.

These improvements in the communication module of the Portenta Machine Control Library are part of the effort to make the library more user-friendly and aligned with best practices in software design.

Temperature Sensors

The Portenta Machine Control Library's latest update has restructured the handling of temperature sensors, dividing the functionalities into two distinct classes for improved clarity and usability. The two separate classes are the following:

- 1 **Temperature RTD:**
Specifically for

Temperature
Detectors (RTD).

- 2 **Temperature TC:**
Dedicated to
Thermocouple (TC)
temperature sensors.

This separation ensures a more organized and intuitive approach to temperature sensing, allowing users to work more efficiently with the specific type of sensor they are using.

Example of Temperature Sensor Usage

Old approach:

```
1 // Using the previous
2 #include <Arduino_Mach
3 using namespace machin
4 void setup() {
5     temp_probes.tc.beg
6     temp_probes.enable
7 }
```

New approach:

```
1 // Using the updated I
2 #include <Arduino_Por
3 void setup() {
4     MachineControl_TCT
5 }
```

In the new approach the initialization of the thermocouple temperature probe is done directly through the

need for accessing a general temperature probe object (`temp_probes`) and its specific member for thermocouples, streamlining the code and making it more straightforward.

These enhancements in managing temperature sensors within the Portenta Machine Control Library simplify integrating and utilizing temperature sensors in industrial applications.

Conclusion

In this tutorial, we have explored the features of the `Arduino_PortentaMachineControl`

library, specifically designed for the Portenta Machine Control. Covering everything from the library's enhanced structure to its modules like I/O, encoders, and communication interfaces, our goal has been to provide you with a thorough understanding of these improvements.

Whether you're a newcomer or an experienced Arduino user, these insights are designed to improve your ability to implement this library effectively in your projects. We encourage you to experiment with what you've learned here and share your experiences with the Arduino community. Thank you for following this guide, and we hope it has been a valuable

Arduino. Your feedback is always welcome!

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

